# sub-agent系统

## 目录


- 第一章：工程总览与文件地图
- 第二章：类型系统与数据结构
- 第三章：Agent 创建与启动流程
- 第四章：消息循环与查询引擎
- 第五章：工具过滤与权限体系
- 第六章：Fork Subagent 算法
- 第七章：内置 Agent 详解
- 第八章：持久化记忆系统
- 第九章：任务状态机与生命周期
- 第十章：隔离机制
- 第十一章：协调者多 Agent 编排
- 第十二章：数据流全景图
- 第十三章：关键算法伪代码实现
- 第十四章：设计模式与架构哲学
- 第十五章：安全性与边界控制
- 附录：常量速查表


---

# 第一章：文件地图

## 1.1 文件职责速查表

| 文件 | 行数 | 核心职责 | 关键导出 |
|---|---|---|---|
| `AgentTool.tsx` | 1398 | 入口：参数解析、路由判断、结果渲染 | `AgentTool.call()`，`AgentToolComponent` |
| `runAgent.ts` | 974 | 执行循环：上下文初始化、query 迭代、cleanup | `runAgent()` |
| `agentToolUtils.ts` | 687 | 工具过滤、结果聚合、异步生命周期 | `filterToolsForAgent()`，`runAsyncAgentLifecycle()` |
| `loadAgentsDir.ts` | 756 | Agent 定义解析：md/json/插件 | `getAgentDefinitionsWithOverrides()` |
| `forkSubagent.ts` | 211 | Fork 逻辑：上下文继承、消息克隆 | `buildForkedMessages()` |
| `prompt.ts` | 288 | 系统提示词构建 | `getPrompt()` |
| `LocalAgentTask.tsx` | 683 | 任务状态机、进度追踪、通知入队 | `registerAsyncAgent()`，`enqueueAgentNotification()` |
| `forkedAgent.ts` | 690 | Fork 专用 context 创建、usage 追踪 | `runForkedAgent()`，`createSubagentContext()` |
| `coordinatorMode.ts` | 370 | 协调者系统提示词、worker 工具上下文 | `getCoordinatorSystemPrompt()` |
| `agentMemory.ts` | 178 | 记忆目录管理、安全路径验证 | `loadAgentMemoryPrompt()` |

## 1.2 调用层次

```
User Input
  └─ AgentTool.call()              [AgentTool.tsx:1398]
       ├─ resolveAgentType()       → 确定 agent 类型
       ├─ validateMcpRequirements()→ 检查 MCP 依赖
       ├─ setupIsolation()         → Worktree / Remote
       ├─ determineSyncAsync()     → 同步 or 异步
       │
       ├─ [异步分支]
       │    ├─ registerAsyncAgent()        [LocalAgentTask.tsx]
       │    └─ runAsyncAgentLifecycle()    [agentToolUtils.ts]
       │         └─ runAgent() → query()
       │
       └─ [同步分支]
            ├─ registerAgentForeground()  [LocalAgentTask.tsx]
            └─ runAgent()                 [runAgent.ts]
                 ├─ createSubagentContext()   [forkedAgent.ts]
                 ├─ initializeAgentMcpServers()
                 ├─ preloadSkills()
                 └─ for await message of query()
                      └─ yield messages → 父 session
```

---

# 第二章：类型系统与数据结构

## 2.1 AgentTool 调用输入

```python
// AgentTool.tsx: Input Zod Schema
const AgentToolInput = z.object({
  description:  z.string().describe("Agent task description"),
  prompt:       z.string().describe("Instructions for the agent"),
  subagent_type: z.string().optional()
                  .describe("Specialized agent type: Explore | Plan | general-purpose"),
  model:        z.enum(["sonnet", "opus", "haiku"]).optional()
                  .describe("Model override"),
  run_in_background: z.boolean().optional()
                  .describe("Run in background (async)"),
  name:         z.string().optional()
                  .describe("Agent name for SendMessage routing"),
  team_name:    z.string().optional()
                  .describe("Agent team for multi-agent spawning"),
  mode:
z.enum(["bubble","plan","acceptEdits","bypassPermissions","auto"]).optional(),
  isolation:     z.enum(["worktree","remote"]).optional()
                  .describe("Execution isolation mode"),
  cwd:          z.string().optional()
                  .describe("Override working directory"),
});
```

## 2.2 Agent 定义完整类型

```typescript
// loadAgentsDir.ts: BaseAgentDefinition
interface BaseAgentDefinition {
  agentType: string;                          // 唯一标识符
  whenToUse: string;                           // 使用场景（供 model 决策）
  tools?: string[];                            // 允许的工具列表，['*']=全部
  disallowedTools?: string[];                  // 禁止的工具列表
  skills?: string[];                           // 预加载的 skill 列表
  mcpServers?: AgentMcpServerSpec[];           // Agent 专属 MCP 服务器
  hooks?: HooksSettings;                       // Session hooks 配置
  color?: AgentColorName;                      // UI 颜色
  model?: string | 'inherit';                 // 模型覆盖，inherit=继承父
  effort?: EffortValue;                       // 努力程度标记
  permissionMode?: PermissionMode;             // 权限模式
  maxTurns?: number;                           // 最大对话轮数
  filename?: string;                           // 来源文件名
  baseDir?: string;                            // 来源目录
  criticalSystemReminder_EXPERIMENTAL?: string; // 关键系统提醒
  requiredMcpServers?: string[];              // 必须的 MCP 服务器
  background?: boolean;                        // 强制后台运行
  initialPrompt?: string;                     // 初始提示词
  memory?: AgentMemoryScope;                   // 记忆作用域
  isolation?: 'worktree' | 'remote';          // 隔离模式
  pendingSnapshotUpdate?: { snapshotTimestamp: string }; // 快照更新
  omitClaudeMd?: boolean;                     // 跳过 CLAUDE.md 注入
}

// 三种 Agent 来源

// 第一类是CC的内置agent，包括:
// 始终开启: generalPurpose、statuslineSetup
// 条件开启: explore、plan、claudeCodeGuide、verification
// 当 COORDINATOR_MODE 环境变量开启时，这 6 个 Agent 会被 getCoordinatorAgents() 替代，作为协调模式下的 worker agents 使用
interface BuiltInAgentDefinition extends BaseAgentDefinition {
  source: 'built-in';
  baseDir: 'built-in';
  getSystemPrompt(params: AgentSystemPromptParams): string;
  callback?(params: AgentCallbackParams): Promise<void> | void;
}

// 第二类是用户/项目/策略层面的配置文件定义的 Agent
interface CustomAgentDefinition extends BaseAgentDefinition {
  source: SettingSource; // 'userSettings' | 'projectSettings' | 'policySettings' | 'flagSettings'
  getSystemPrompt(): string;
  filename?: string;
  baseDir?: string;
}

// 第三类是外部插件系统注册的 Agent
interface PluginAgentDefinition extends BaseAgentDefinition {
  source: 'plugin';
  getSystemPrompt(): string;
```

```
  filename?: string;
  plugin: string;
}
```

## 2.3 Agent 工具调用结果

```
// agentToolUtils.ts: AgentToolResult (Zod Schema)
const AgentToolResult = z.object({
  agentId: z.string(), // 本轮运行的 Agent 实例 ID
  agentType: z.string().optional(), // Agent 类型名称（如 'explore', 'plan' 等）
  content: z.array(z.object({ // Agent 返回的内容，type 目前固定为 'text'
    type: z.literal('text'),
    text: z.string(), // Agent 的文本输出
  })),
  totalToolUseCount: z.number(), // 本轮 Agent 内部工具调用的总次数
  totalDurationMs: z.number(), // 本轮 Agent 运行的总耗时（毫秒）
  totalTokens: z.number(), // 本轮消耗的 token 总数（input + output）
  usage: z.object({ // API 返回的详细用量数据
    input_tokens: z.number(), // 本次请求的输入 token 数
    output_tokens: z.number(), // 本次请求的输出 token 数
    cache_creation_input_tokens: z.number().nullable(), // 创建持久缓存时消耗的输入 token（部分模型支持）
    cache_read_input_tokens: z.number().nullable(), // 读取已存在的缓存上下文时节省的 token 数
    server_tool_use: z.object({ // 服务端执行的工具调用统计
      web_search_requests: z.number(), // 网页搜索请求次数
      web_fetch_requests: z.number(), // 网页抓取请求次数
    }).nullable(),
    service_tier: z.enum(['standard', 'priority', 'batch']).nullable(), // API 服务等级
    cache_creation: z.object({ // 短期缓存（ephemeral cache）的用量
      ephemeral_1h_input_tokens: z.number(), // TTL 约 1 小时的缓存消耗 token
      ephemeral_5m_input_tokens: z.number(), // TTL 约 5 分钟的缓存消耗 token
    }).nullable(),
  }),
});
```

## 2.4 任务状态

```
// LocalAgentTask.tsx: LocalAgentTaskState
interface LocalAgentTaskState {
  type: 'local_agent';                          // 状态机类型标记
  agentId: string;                              // 全局唯一 ID
  prompt: string;                               // 任务描述
  selectedAgent?: AgentDefinition;              // 关联的 Agent 定义
  agentType: string;                            // Agent 类型字符串
  model?: string;                               // 使用的模型
  abortController?: AbortController;            // 中止控制器
  unregisterCleanup?: () => void;               // 注销清理函数
  error?: string;                               // 错误信息
  result?: AgentToolResult;                     // 最终结果
  progress?: AgentProgress;                     // 当前进度
  retrieved: boolean;                           // 是否已被 UI 读取
  messages?: Message[];                         // 消息历史（内存缓存）
```

```
  lastReportedToolCount: number;                // 上次上报的工具调用数
  lastReportedTokenCount: number;               // 上次上报的 token 数
  isBackgrounded: boolean;                      // 是否已转入后台
  pendingMessages: string[];                    // 待投递消息队列
  retain: boolean;                              // UI 是否持有（防 GC）
  diskLoaded: boolean;                          // bootstrap 是否完成
  evictAfter?: number;                          // GC 截止时间戳
}

// 进度数据结构
interface ToolActivity {
  toolName: string;
  input: Record<string, unknown>;
  activityDescription?: string;
  isSearch?: boolean;       // 是否为搜索工具（低强度活动）
  isRead?: boolean;        // 是否为只读工具
}

interface AgentProgress {
  toolUseCount: number;
  tokenCount: number;
  lastActivity?: ToolActivity;
  recentActivities?: ToolActivity[];  // 最多保留 5 条
  summary?: string;                   // 周期性摘要
}

// 进度追踪器（运行时）
interface ProgressTracker {
  toolUseCount: number;
  latestInputTokens: number;            // 累积输入 token（取最新值）
  cumulativeOutputTokens: number;        // 累积输出 token
  recentActivities: ToolActivity[];    // 最近活动（最多 5 条）
}
```

## 2.5 Fork 专用数据结构

```
// forkedAgent.ts: CacheSafeParams
interface CacheSafeParams {
  systemPrompt: SystemPrompt;
  userContext: { [k: string]: string };
  systemContext: { [k: string]: string };
  toolUseContext: ToolUseContext;
  forkContextMessages: Message[];
}

// forkedAgent.ts: ForkedAgentParams
interface ForkedAgentParams {
  promptMessages: Message[];
  cacheSafeParams: CacheSafeParams;
  canUseTool: CanUseToolFn;
  querySource: QuerySource;
  forkLabel: string;
  overrides?: SubagentContextOverrides;
```

```
  maxOutputTokens?: number;
  maxTurns?: number;
  onMessage?: (message: Message) => void;
  skipTranscript?: boolean;
  skipCacheWrite?: boolean;
}

// forkedAgent.ts: SubagentContextOverrides
interface SubagentContextOverrides {
  options?: ToolUseContext['options'];
  agentId?: string;
  agentType?: string;
  messages?: Message[];
  readFileState?: ToolUseContext['readFileState'];
  abortController?: AbortController;
  getAppState?: ToolUseContext['getAppState'];
  shareSetAppState?: boolean;         // 共享父的 setAppState
  shareSetResponseLength?: boolean;   // 共享父的 setResponseLength
  shareAbortController?: boolean;    // 共享父的 AbortController
  criticalSystemReminder_EXPERIMENTAL?: string;
  requireCanUseTool?: boolean;
  contentReplacementState?: ContentReplacementState;
}
```

### 2.6 权限模式枚举

```
type PermissionMode =
  | 'bubble'           // 权限请求冒泡到父 terminal
  | 'plan'             // 执行前需要生成计划并审批
  | 'acceptEdits'      // 自动批准文件修改
  | 'bypassPermissions'// 跳过所有权限检查
  | 'auto';            // 默认行为
```

### 2.7 记忆作用域

```
type AgentMemoryScope = 'user' | 'project' | 'local';
// user    → ~/.claude/agent-memory/<agentType>/      跨项目全局
// project → <cwd>/.claude/agent-memory/<agentType>/   团队共享
// local   → <cwd>/.claude/agent-memory-local/<agentType>/  仅本地
```

---

## 第三章：Agent 创建与启动流程

### 3.1 完整决策树

```
AgentTool.call(input, toolUseContext, canUseTool, assistantMessage, onProgress?)
│
├─ [Step 1] Permission Check
│   └─ canUseTool('Agent') → 验证用户是否被授权使用 Agent 工具
│
├─ [Step 2] Multi-Agent Spawn? (team_name + name)
│   └─ Yes → spawnTeammate() → 跳过 AgentTool 逻辑，生成 teammate_spawned
```

```
│
├─ [Step 3] Resolve Agent Type  ←──────────────────────────────────┐
│   ├─ subagent_type 已指定 → 使用该类型                              │
│   ├─ forkEnabled + 无 type                                          │
│   │   └─ 是 Fork Subagent → type = 'fork'                          │
│   └─ 均无 → general-purpose（默认）                                 │
│                                                                     │
├─ [Step 4] Recursive Fork Guard                                      │
│   └─ querySource === 'agent:builtin:fork' → 拒绝，防止无限递归      │
│                                                                     │
├─ [Step 5] MCP Requirements Validation                               │
│   ├─ 有 requiredMcpServers 的 agent                                 │
│   │   ├─ 等待 pending 服务器最多 30 秒                              │
│   │   └─ validateMcpRequirements() → 缺失则报错                     │
│   └─ 无要求 → 跳过                                                  │
│                                                                     │
├─ [Step 6] Isolation Setup                                           │
│   ├─ isolation = 'remote' → delegate to CCR → remote_launched       │
│   └─ isolation = 'worktree' → create git worktree                   │
│                                                                     │
├─ [Step 7] System Prompt Setup                                       │
│   ├─ Fork path → inherit parent's rendered system prompt            │
│   └─ Normal path → getAgentSystemPrompt() → 构建新 prompt           │
│                                                                     │
├─ [Step 8] Sync/Async Decision  ──────────────────────────────────┘
│   │
│   ├─ [同步分支] run_in_background = false
│   │   ├─ registerAgentForeground() → 注册任务，开始倒计时自动后台
│   │   ├─ onProgress hint after 2s (PROGRESS_THRESHOLD_MS)
│   │   ├─ race: 下一条消息 vs 后台信号
│   │   │   ├─ 消息先到 → finalizeResult(), unregister → 返回
│   │   │   └─ 信号先到 → backgroundAgentTask() → async_launched
│   │   └─ 错误处理 → partialResult 恢复，AbortError 上抛
│   │
│   └─ [异步分支] run_in_background = true (或 coordinator/fork)
│       ├─ registerAsyncAgent() → agentId, task state = pending
│       ├─ agentNameRegistry[name] = agentId (if named)
│       ├─ runAsyncAgentLifecycle() 在后台启动
│       └─ 返回 { status: 'async_launched', agentId, outputFile, ... }
│
└─ [Step 9] Cleanup (finally 块)
    ├─ mcpCleanup()
    ├─ clearSessionHooks()
    ├─ killShellTasksForAgent()
    └─ killMonitorMcpTasksForAgent()
```

## 3.2 Agent 类型路由表

| `subagent_type` | Agent 定义来源 | 执行模式 | 默认工具 |
|---|---|---|---|
| `"Explore"` | 内置 | 同步 one-shot | Read-only |
| `"Plan"` | 内置 | 同步 one-shot | Read-only |
| `"general-purpose"` | 内置 | 同步/异步 | 全部允许 |

| subagent_type | Agent 定义来源 | 执行模式 | 默认工具 |
|---|---|---|---|
| "worker" | 内置 (coordinator) | 异步 | 受限集 |
| "fork" | 虚拟类型 (Fork) | 异步 | 继承父 |
| 用户自定义名称 | md/json/插件 | 同步/异步 | 按定义 |
| 未指定 | general-purpose | 同步→异步 | 全部允许 |

## 3.3 异步 vs 同步决策算法

```
// AgentTool.tsx: determineSyncAsync 逻辑
function shouldRunAsync(options: AgentToolInput, agentType: string): boolean {
  // 明确要求后台
  if (options.run_in_background) return true;

  // 协调者模式下的子 agent
  if (isCoordinatorMode()) return true;

  // Fork subagent
  if (isForkSubagentEnabled() && !options.subagent_type) return true;

  // Agent 定义要求后台
  const agentDef = getAgentDefinition(agentType);
  if (agentDef?.background) return true;

  return false;
}
```

## 3.4 自动后台机制

同步 Agent 有 2 秒延迟提示（`PROGRESS_THRESHOLD_MS = 2000`），之后自动转入后台：

```
// 同步分支
const { taskId, backgroundSignal, cancelAutoBackground } =
  registerAgentForeground({ agentId, prompt, agentType, model, abortController });

// 2 秒后弹出后台提示
const autoBackgroundTimer = setTimeout(() => {
  // 父 session UI 显示后台提示
  showBackgroundHint(agentType, agentId, outputFile);
}, PROGRESS_THRESHOLD_MS);

// 竞速: 消息完成 vs 后台信号
const raceResult = await Promise.race([
  waitForNextMessage(),
  backgroundSignal,
]);

if (raceResult === 'backgrounded') {
  // 后台信号先到 → 转为异步
  backgroundAgentTask(taskId, getAppState, setAppState);
  return { status: 'async_launched', agentId, outputFile, ... };
} else {
```

```
  // 消息完成
  cancelAutoBackground();
  return finalizeResult(messages);
}
```

---

# 第四章：消息循环与查询引擎

## 4.1 runAgent() 完整执行流程

```
// runAgent.ts: runAgent() 伪代码
async function* runAgent(params: RunAgentParams): AsyncGenerator<Message> {

  // ── Phase 1: 初始化 ──────────────────────────────────────────
  const agentDef = getAgentDefinition(params.agentType);
  const model = resolveAgentModel(agentDef, params.model);

  // 生成 agentId（如未提供）
  const agentId = params.agentId ?? generateAgentId();

  // 注册性能追踪
  Perfstart('agent', agentId, agentDef.agentType);

  // ── Phase 2: 构建上下文 ──────────────────────────────────────
  // 2a. 构建 user/system context
  const { userContext, systemContext } = buildAgentContext({
    omitClaudeMd: agentDef.omitClaudeMd,
    cwd: params.cwd,
  });

  // 2b. 构建权限上下文
  const permissionContext = buildPermissionContext({
    parentMode: toolUseContext.options?.permissionMode,
    agentMode: agentDef.permissionMode,
    // 特殊规则: async/bypassPermissions 下避免权限弹窗
    shouldAvoidPermissionPrompts:
      params.isAsync ||
      agentDef.permissionMode === 'bypassPermissions',
    awaitAutomatedChecksBeforeDialog:
      !params.isAsync && agentDef.permissionMode === 'acceptEdits',
  });

  // 2c. 解析工具集
  const tools = params.useExactTools
    ? toolUseContext.tools  // Fork 路径: 完全继承父的工具
    : resolveAgentTools(agentDef, availableTools, params.isAsync);

  // 2d. 构建系统提示词
  const systemPromptLines = await getAgentSystemPrompt(
    agentDef, toolUseContext, model, dirs, tools
  );

  // ── Phase 3: 前置 Hook ───────────────────────────────────────
```

```
// 3a. 执 行 SubagentStart hooks
await executeHooks('SubagentStart', { agentDef, agentId });

// 3b. 注册 frontmatter hooks（来自 agent 定义）
registerFrontmatterHooks(agentDef.hooks);

// 3c. 预加载 skills
await preloadSkills(agentDef.skills ?? [], toolUseContext);

// ── Phase 4: Agent MCP 服务器 ──────────────────────────────────
const { clients: agentMcpClients, tools: agentMcpTools, cleanup: mcpCleanup } =
  await initializeAgentMcpServers(agentDef, toolUseContext.mcpClients);

// ── Phase 5: 创建隔离上下文 ────────────────────────────────────
const subagentContext = createSubagentContext(toolUseContext, {
  agentId,
  agentType: agentDef.agentType,
  // AbortController 链接到父
  abortController: createChildAbortController(
    toolUseContext.abortController,
    agentId
  ),
  // Async agent 不能控制父 UI
  setAppState: undefined,
  setResponseLength: undefined,
  // 读取状态缓存克隆（保留父的缓存命中决策）
  readFileState: toolUseContext.readFileState,
});

// ── Phase 6: 记录初始转录 ──────────────────────────────────────
recordSidechainTranscript({
  type: 'metadata',
  agentId,
  agentType: agentDef.agentType,
  model,
  systemPromptLines,
  startTime: Date.now(),
});

// ── Phase 7: 主查询循环 ────────────────────────────────────────
try {
  for await (const message of query({
    // ... query options
  })) {
    // 转发 TTFT/OTPS 指标
    if (message.stream_event?.type === 'stream_request_start') {
      const { ttft_ms, first_token_ms } = message.stream_event;
      forwardMetric('time_to_first_token_ms', ttft_ms);
      forwardMetric('output_tokens_per_second', calculateOTPS(first_token_ms));
    }

    // 过滤不完整的 tool_use（尾部截断保护）
    if (message.type === 'assistant' && isMaxTurnsReached(message)) {
```

```
        yield* emitIncompleteToolCallWarning(message, tools);
      }

      // 记录到转录文件
      if (isRecordableMessage(message)) {
        recordSidechainTranscript(message);
      }

      // 追加到 agentMessages（用于最终结果）
      agentMessages.push(message);

      yield message;  // 发送给父 session
    }
  }

  // ── Phase 8: 清理 ──────────────────────────────────────────────
  finally {
    await mcpCleanup();                        // 断开 Agent MCP
    clearSessionHooks();                       // 清除 Hooks
    releasePromptCacheTracking(agentId);       // 释放 Prompt Cache
    releaseFileStateCache();                  // 释放文件读取缓存
    Perfstop('agent', agentId);               // 停止性能追踪
    clearSubagentTranscriptDir();             // 清除转录临时文件
    removeTodoEntry(agentId);                 // 清除 Todo 条目
    killShellTasksForAgent(agentId);          // 杀死 Agent 的 Shell
    killMonitorMcpTasksForAgent(agentId);     // 停止 MCP 监控任务
  }
}
```

## 4.2 forkSubagent 消息构建算法

```
// forkSubagent.ts: buildForkedMessages()
// 目标: 构建与父 API 请求字节级一致的消息序列
// 目的: 最大化 API cache hit（response_prefix 可复用）

function buildForkedMessages(
  directive: string,          // 子 agent 的任务指令
  assistantMessage: AssistantMessage  // 父的最后一条 assistant 消息
): MessageType[] {
  // Step 1: 深拷贝避免污染父状态
  const clonedAssistant = deepClone(assistantMessage);

  // Step 2: 收集所有 tool_use blocks
  const toolUses = clonedAssistant.content
    .filter((block): block is ToolUseBlock => block.type === 'tool_use');

  // Step 3: 为每个 tool_use 创建完全相同的占位结果
  // 关键: 文本必须完全相同 → API 才能命中 cache
  const toolResultBlocks: ToolResultBlock[] = toolUses.map(toolUse => ({
    type: 'tool_result' as const,
    tool_use_id: toolUse.id,
    content: [{
      type: 'text' as const,
```

```
      text: FORK_PLACEHOLDER_RESULT,  // ≡ 'Fork started — processing in background'
      is_error: false,
    }],
  }));

  // Step 4: 构建用户指令消息
  // 注意: 此消息是新增的 (不参与 cache), 但不影响 cache 命中率
  const userDirectiveMessage: Message = {
    type: 'user',
    content: [{
      type: 'text',
      text: buildChildMessage(directive),  // 包含 10 条规则
    }],
  };

  // Step 5: 组装最终消息序列
  return [
    clonedAssistant,            // 1. 父的 assistant (含 tool_use blocks)
    ...toolResultBlocks,         // 2. N 个完全相同的 placeholder tool_result
    userDirectiveMessage,        // 3. 指令消息 (触发子 agent 执行)
  ];
}

// buildChildMessage: 生成 XML 指令块
function buildChildMessage(directive: string): string {
  return `<${FORK_BOILERPLATE_TAG}>
  Task: ${directive}
  Rules:
  1. Continue from the previous agent's last action
  2. Complete the task autonomously — do not re-confirm
  3. Report completion with a concise summary
  4. For multi-step: execute all remaining steps
  5. One-shot: run to completion without asking
  6. Tool restrictions override your default capabilities
  7. If blocked: attempt alternatives, report if impossible
  8. Do not explain your reasoning unless asked
  9. Result goes to parent session — avoid redundant context
  10. You control execution — parent is waiting for result
</${FORK_BOILERPLATE_TAG}>`;
}
```

## 4.3 异步 Agent 生命周期

```
// agentToolUtils.ts: runAsyncAgentLifecycle()
async function runAsyncAgentLifecycle(params: {
  agentId: string;
  toolUseContext: ToolUseContext;
  onProgress?: (message: Message) => void;
  enableSummarization?: boolean;
  ...others
}): Promise<void> {

  // Step 1: 创建进度追踪器
```

```
const tracker = createProgressTracker();
const resolveActivityDescription = createActivityDescriptionResolver(
  toolUseContext.tools ?? []
);

const agentMessages: Message[] = [];

// Step 2: 主循环 —— 消费 runAgent() 的消息流
try {
  for await (const message of runAgent({ agentId, ... })) {
    // 保存消息历史
    agentMessages.push(message);

    // 持久化到磁盘（用于 Bootstrap / TaskOutput）
    if (message.type === 'assistant' || message.type === 'progress') {
      rootSetAppState(draft => {
        const task = draft.localAgentTasks[agentId];
        if (task?.retain) {
          task.messages = [...(task.messages ?? []), message];
        }
      });
    }

    // 更新进度追踪器
    updateProgressFromMessage(tracker, message, resolveActivityDescription);

    // 发送进度事件到父 session
    emitTaskProgress(
      tracker, agentId,
      message.type === 'assistant'
        ? getLastToolUseId(message)
        : undefined,
      resolveActivityDescription(message),
      startTime,
      getLastToolUseName(message)
    );

    // 回调通知
    params.onProgress?.(message);
  }

  // Step 3: 构建结果
  const result = finalizeAgentTool(agentMessages, agentId, {
    startTime,
    totalToolUseCount: tracker.toolUseCount,
    // ... usage metrics
  });

  // Step 4: 停止摘要生成（如果启用）
  if (params.enableSummarization) {
    await stopAgentSummarization(agentId);
  }
```

```
    // Step 5: 标记完成（先标记，确保状态一致性）
    completeAgentTask(result, setAppState);

    // Step 6: 安全审查
    await classifyHandoffIfNeeded({ agentId, result, toolUseContext });

    // Step 7: 发送通知
    enqueueAgentNotification({
      agentId,
      toolUseId: params.toolUseId,
      status: 'completed',
      result,
      worktree: params.getWorktreeResult?.(),
      usage: result.usage,
    });

  } catch (error) {
    if (isAbortError(error)) {
      // 用户中止：发送 killed 通知（不是 failed）
      enqueueAgentNotification({
        agentId, status: 'killed',
        result: extractPartialResult(agentMessages),
      });
    } else {
      // 真实错误
      failAgentTask(agentId, String(error), setAppState);
      enqueueAgentNotification({
        agentId, status: 'failed',
        result: extractPartialResult(agentMessages),
        error: String(error),
      });
    }
  } finally {
    // Step 8: 清理
    await clearInvokedSkills(agentId);
    await dumpState(agentId);
  }
}
```

---

# 第五章：工具过滤与权限体系

## 5.1 六层过滤算法

```
// agentToolUtils.ts: filterToolsForAgent()
// 输入：全部工具集、agent 定义、是否异步、权限模式
// 输出：过滤后的工具集

function filterToolsForAgent(
  allTools: Tools,
  agentDef: AgentDefinition,
  isAsync: boolean = false,
  permissionMode: PermissionMode = 'auto'
```

```
): Tools {

  let result = allTools;

  // ── Layer 1: 全局禁止列表 ──────────────────────────────────────
  // ALL_AGENT_DISALLOWED_TOOLS: 对所有 agent 类型都禁用
  result = result.exclude(ALL_AGENT_DISALLOWED_TOOLS);
  // Agent 嵌套例外: 仅在 ant 内部版本中允许（防止外部用户递归调用）
  if (process.env.USER_TYPE !== 'ant') {
    result = result.exclude('Agent');
  }

  // ── Layer 2: 异步 Agent 强制白名单 ────────────────────────────
  // 异步后台 agent 不能使用危险工具
  if (isAsync) {
    result = result.filter(t =>
      ASYNC_AGENT_ALLOWED_TOOLS.has(t.name)
    );
  }

  // ── Layer 3: 自定义 Agent 额外禁止 ────────────────────────────
  // 自定义 agent（md/json 配置）额外禁止任务管理工具
  if (agentDef.source === 'userSettings' ||
      agentDef.source === 'projectSettings') {
    result = result.exclude(CUSTOM_AGENT_DISALLOWED_TOOLS);
  }

  // ── Layer 4: Agent 定义允许列表 ───────────────────────────────
  // ['*'] 表示全部允许（但仍受 Layer 1/2/3 限制）
  if (agentDef.tools && !agentDef.tools.includes('*')) {
    result = result.filter(t => agentDef.tools!.includes(t.name));
  }

  // ── Layer 5: Agent 定义禁止列表（优先级更高）──────────────────
  if (agentDef.disallowedTools) {
    result = result.exclude(agentDef.disallowedTools);
  }

  // ── Layer 6: 权限模式调整 ─────────────────────────────────────
  switch (permissionMode) {
    case 'bypassPermissions':
      // 移除所有权限检查包装器
      result = result.map(t => t.removePermissionWrapper());
      break;
    case 'acceptEdits':
      // 确保 Edit/Write 工具可用（但保留确认弹窗）
      result = result.map(t => {
        if (t.name === 'Edit' || t.name === 'Write') {
          return t.withAutoApprove();
        }
        return t;
      });
      break;
```

```
    case 'plan':
      // 所有工具都需要先出 plan
      result = result.map(t => t.wrapWithPlanApproval());
      break;
  }

  return result;
}
```

## 5.2 工具禁用集合详解

```
// constants/tools.ts

// ── 所有 Agent 一律禁用 ──────────────────────────────────────────
const ALL_AGENT_DISALLOWED_TOOLS = new Set([
  TASK_OUTPUT_TOOL_NAME,       // 禁止等待其他任务（防止死锁）
  EXIT_PLAN_MODE_V2_TOOL_NAME, // 禁止主动退出 plan mode
  ENTER_PLAN_MODE_TOOL_NAME,  // 禁止主动进入 plan mode
  AGENT_TOOL_NAME,             // 禁止嵌套 Agent（除非 ant 内部）
  ASK_USER_QUESTION_TOOL_NAME, // 禁止向用户提问（单向通信）
  TASK_STOP_TOOL_NAME,        // 禁止停止任务
  WORKFLOW_TOOL_NAME,         // 禁止触发 workflow（如果功能开启）
]);

// ── 异步后台 Agent 仅允许 ────────────────────────────────────────
const ASYNC_AGENT_ALLOWED_TOOLS = new Set([
  // 文件操作
  'Read', 'Edit', 'Write', 'NotebookEdit',
  // 搜索工具
  'Grep', 'Glob', 'WebSearch', 'WebFetch',
  // Shell 执行
  'Bash', ...ALL_SHELL_TOOL_NAMES,
  // 任务管理
  'TodoWrite', 'TodoGet', 'TodoList', 'TodoUpdate',
  // 其他
  'Skill', 'SyntheticOutput',
  'ToolSearch',
  'EnterWorktree', 'ExitWorktree',
]);

// ── 队内 Agent（In-process Teammate）额外允许 ──────────────────
const IN_PROCESS_TEAMMATE_ALLOWED_TOOLS = new Set([
  ...IN_ASYNC_AGENT_ALLOWED_TOOLS,
  'TaskCreate', 'TaskGet', 'TaskList', 'TaskUpdate', 'TaskDelete',
  'SendMessage',
  // Cron 工具（如果 AGENT_TRIGGERS 功能开启）
  ...(FEATURES.AGENT_TRIGGERS ? ['CronCreate', 'CronDelete', 'CronList'] : []),
]);

// ── 协调者模式 Agent ─────────────────────────────────────────────
const COORDINATOR_MODE_ALLOWED_TOOLS = new Set([
  'Agent', 'TaskStop', 'SendMessage', 'SyntheticOutput',
  'Read', 'Write', 'Edit', 'Bash',
```

```
  'Grep', 'Glob', 'WebSearch', 'WebFetch',
  // ... 大部分工具都允许
]);
```

### 5.3 工具过滤时机与 resolveAgentTools

```
// agentToolUtils.ts: resolveAgentTools()
// 与 filterToolsForAgent 配合使用，处理通配符展开

function resolveAgentTools(
  agentDef: AgentDefinition,
  availableTools: Tools,
  isAsync?: boolean,
  isMainThread?: boolean,
): ResolvedAgentTools {

  // Case 1: ['*'] 通配符 → 展开为全部可用工具
  if (agentDef.tools?.includes('*')) {
    return {
      hasWildcard: true,
      validTools: availableTools.map(t => t.name),
      invalidTools: [],
      resolvedTools: filterToolsForAgent(
        availableTools, agentDef, isAsync, permissionMode
      ),
      allowedAgentTypes: undefined,
    };
  }

  // Case 2: 显式列表 → 验证每个工具是否存在
  const validTools: string[] = [];
  const invalidTools: string[] = [];

  for (const toolName of (agentDef.tools ?? [])) {
    if (availableTools.has(toolName)) {
      validTools.push(toolName);
    } else {
      invalidTools.push(toolName);  // 工具不存在 → 警告
    }
  }

  return {
    hasWildcard: false,
    validTools,
    invalidTools,
    resolvedTools: filterToolsForAgent(validTools, agentDef, isAsync),
    allowedAgentTypes: undefined,
  };
}
```

---

## 第六章：Fork Subagent 算法

## 6.1 Fork vs Normal Subagent 核心区别

| 维度 | Normal Subagent | Fork Subagent |
|---|---|---|
| 触发条件 | 显式指定 `subagent_type` | `forkEnabled=true` + 无 type |
| 系统提示词 | 新构建（重新 tokenize） | 字节级继承（可命中 cache） |
| 工具集 | 重新过滤 | `useExactTools=true` 完全继承 |
| 消息历史 | 仅 `prompt` | 完整父对话历史 |
| API cache 命中率 | 低（每次都是新 prefix） | 高（response_prefix 可复用） |
| 执行模式 | 同步或异步 | 强制异步 |
| 工具调用序列化 | 重新序列化 | 继承父的序列化（byte-identical） |
| 适用场景 | 完全不同任务 | 同一任务的延续/细分 |

## 6.2 递归深度守卫

```
// forkSubagent.ts: isInForkChild()
// 防止 fork subagent 再 fork（无限递归）

function isInForkChild(messages: MessageType[]): boolean {
  // 扫描消息历史，查找 FORK_BOILERPLATE_TAG XML 标记
  // 如果存在，说明当前在 fork child 的上下文中
  return messages.some(msg => {
    if (msg.type !== 'user') return false;
    const text = extractTextContent(msg);
    return text.includes(`<${FORK_BOILERPLATE_TAG}>`);
  });
}

// 调用点：在 AgentTool.call() 的 Step 4 中
if (
  toolUseContext.querySource === 'agent:builtin:fork' ||
  isInForkChild(assistantMessage)
) {
  throw new Error(
    `Cannot spawn fork sub-agent: Already in fork child context. ` +
    `Fork subagents cannot be nested.`
  );
}
```

**Note**

子 Agent 通过 fork 机制创建时，会完整继承父 Agent 的所有工具能力，其中也包括 Agent 工具本身，因此子 Agent 理论上具备再次 fork 新子 Agent 的能力——这就会产生无限递归的分身嵌套问题。为防止这种情况，系统在 fork 的消息注入阶段会向子 Agent 的上下文中插入一个唯一的 <FORK_BOILERPLATE_TAG> XML 标记，同时在 Agent 工具的调用入口处设置守卫检查：一旦发现消息历史中已存在该标记（即当前已在 fork child 上下文中），就直接拒绝再次 fork，从而将递归深度锁定为 1

## 6.3 Fork 缓存优化原理

```
┌──────────────────────────────────────────────────────────────────────┐
│                        API Cache 命中机制                            │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  父 Agent 请求:                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ System Prompt Hash (SHA256)  ←── 父的系统提示词              │   │
│  │ Tool Definitions Hash (SHA256) ←── 父的工具序列化            │   │
│  │ Messages Prefix  ←── 父的消息历史                            │   │
│  │ [Assistant Message + tool_use blocks]                        │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                            │                                         │
│                            ▼                                         │
│  API 返回 (cached prefix):                                           │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ Thinking... [完整 thinking 文本，可直接复用]                 │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  Fork Subagent 请求:                                                 │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ System Prompt Hash  ←── 与父完全相同 ✓                       │   │
│  │ Tool Definitions Hash ←── 与父完全相同 ✓                     │   │
│  │ Messages Prefix  ←── 父的 assistant + placeholders ✓         │   │
│  │ [Assistant + tool_result placeholders + directive]           │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                            │                                         │
│                            ▼                                         │
│  API 返回 (cache hit):                                               │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ HTTP 200 + cache: true                                       │   │
│  │ response_prefix_bytes_reused: ~90%                           │   │
│  │ 只需生成新 directive message 的完成部分                      │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

## 6.4 Fork Worktree 隔离

```
// forkSubagent.ts: buildWorktreeNotice()
// 当 fork + worktree 时，向父 session 发送路径通知

function buildWorktreeNotice(
  parentCwd: string,     // /repo
  worktreeCwd: string,    // /repo/.claude/worktrees/agent-abc
): string {
  return [
    `Note: This agent is running in an isolated git worktree:`,
    `  Worktree: ${worktreeCwd}`,
    `  Parent:   ${parentCwd}`,
    ``,
    `File paths in the worktree map to the parent as follows:`,
    `  ${worktreeCwd}/src/file.ts → ${parentCwd}/src/file.ts`,
```

```
    ``,
    `Any changes made here are isolated from the parent branch.`,
    `When complete, these changes will be summarized for you.`,
  ].join('\n');
}
```

**Note**

- 当 fork 子 Agent 时，如果指定了 isolation: 'worktree'（即在独立的 git worktree 中运行），子 Agent 的工作目录会从 /repo 切换到一个隔离的 /repo/.claude/worktrees/agent-abc 目录。此时子 Agent 对文件的读写都在 worktree 副本中进行，和父 session 的主分支完全隔离。
- 为了让父 session 在子 Agent 完成后能理解它操作了哪些文件，系统通过 buildWorktreeNotice() 向父 session 注入一条说明，告知：worktree 中的路径和父目录的路径是如何一一对应的（例如 worktree/src/file.ts 对应 父目录/src/file.ts），以及这些修改是隔离的、不会影响父分支。这样父 session 在收到子 Agent 的结果后，就能准确地将 worktree 中的变更映射回原始路径，汇总给用户。

---

# 第七章：内置 Agent 详解

**Note**

设置sub-agent的几点原因：

- 需要处理**独立多步任务**
- 防止上下文溢出/污染
- 工具与权限的隔离（例如下面的只读模式等）
- 可以并行处理或后台异步处理
- 引入验证与反思机制（例如 `verification`）
- 还有最常见的：角色专业化与提示词聚焦

## 7.1 全部内置 Agent 一览

| 名称 | 文件 | 用途 | 工具策略 | 模式 |
|---|---|---|---|---|
| `general-purpose` | `generalPurposeAgent.ts` | 通用任务 | `['*']` | 同步/异步 |
| `Explore` | `exploreAgent.ts` | 快速搜索 | Read-only | One-shot |
| `Plan` | `planAgent.ts` | 架构设计 | Read-only | One-shot |
| `verification` | `verificationAgent.ts` | 测试验证 | 无编辑 | 异步后台 |
| `claude-code-guide` | `claudeCodeGuideAgent.ts` | 使用帮助 | Read/WebSearch | 同步 |
| `statusline-setup` | `statuslineSetup.ts` | 状态栏 | 全部 | 同步 |

## 7.2 ExploreAgent（快速探索）

```
// built-in/exploreAgent.ts
const EXPLORE_AGENT: BuiltInAgentDefinition = {
  source: 'built-in',
  agentType: 'Explore',
  whenToUse: 'Use when you need to quickly search, read, or explore ' +
    'files, code, documentation, or web content.',
```

```
  tools: ['*'],              // 全部允许（但 ExploreSystemPrompt 限制行为）
  model: isAntInternalUser ? 'inherit' : 'haiku',
  // 关键：跳过 CLAUDE.md 注入，节省上下文
  omitClaudeMd: true,
  maxTurns: 10,              // 限制最大轮数，防止过度探索
  getSystemPrompt() {
    return `You are Claude Code's fast exploration specialist.

KEY BEHAVIOR:
- Be FAST and DIRECT — minimize reasoning, maximize action
- One-shot execution: read, search, or grep, then report
- Avoid unnecessary tool calls — each call must be purposeful
- Report findings concisely — bullet points preferred
- Do NOT edit, write, or modify files
- Do NOT explain your approach unless asked
- If blocked: report what you found and why you're blocked

TOOL USAGE:
- Read: targeted file reads with specific line ranges
- Grep: precise patterns, limit scope, avoid broad searches
- Glob: specific file patterns, not recursive wildcards
- WebSearch/WebFetch: only for external info not in codebase
- Glob + Read is usually faster than Grep for finding files`;
  },
};
```

## 7.3 PlanAgent（架构规划）

```
// built-in/planAgent.ts
const PLAN_AGENT: BuiltInAgentDefinition = {
  source: 'built-in',
  agentType: 'Plan',
  whenToUse: 'Use when you need to design an architecture, ' +
    'plan a refactor, or create a detailed implementation plan.',
  tools: ['*'],              // 全部允许（但 plan prompt 限制修改）
  model: isAntInternalUser ? 'inherit' : 'haiku',
  omitClaudeMd: true,         // 无需 CLAUDE.md 上下文
  maxTurns: 20,               // 规划需要更多轮次
  getSystemPrompt() {
    return `You are Claude Code's architecture and planning specialist.

CRITICAL: READ-ONLY MODE — You must NOT modify any files.

YOUR ROLE:
- Analyze requirements and design solutions
- Create detailed implementation plans (Markdown)
- Identify risks, dependencies, and trade-offs
- Suggest file changes ONLY as text descriptions

OUTPUT FORMAT:
## Analysis
[What the task requires]
```

```
## Proposed Solution
[High-level approach]

## Implementation Plan
### Step 1: [Name]
- File: ...
- Changes: ...
[Continue for each step]

## Risks & Mitigations
[Potential issues and how to address them]

## Alternative Approaches
[Brief alternatives with trade-offs]

TOOL USAGE:
- Read: understand existing code structure
- Grep/Glob: find relevant files
- WebSearch: research best practices or APIs
- WebFetch: reference external documentation`;
  },
};
```

### 7.4 VerificationAgent（验证）

```
// built-in/verificationAgent.ts
const VERIFICATION_AGENT: BuiltInAgentDefinition = {
  source: 'built-in',
  agentType: 'verification',
  whenToUse: 'Use when you need to verify code correctness, ' +
    'run tests, or perform quality assurance checks.',
  tools: ['*'],
  model: 'sonnet',
  background: true,           // 强制后台运行
  maxTurns: 50,
  getSystemPrompt() {
    return `You are Claude Code's verification and QA specialist.

YOUR MISSION:
Think like an adversarial reviewer. Find what could break.

VERIFICATION CATEGORIES (apply all relevant):
1. Frontend: render check, interaction, responsive
2. Backend: API correctness, error handling, edge cases
3. CLI: argument parsing, output format, exit codes
4. Infrastructure: config, secrets, scaling, security
5. Library: API surface, types, exports, documentation
6. Mobile: platform conventions, lifecycle
7. Data/ML: training, inference, data pipeline
8. Database: migrations, queries, indexes, constraints
9. Refactoring: behavior preservation, backward compatibility

REQUIRED OUTPUT FORMAT:
```

```
## Verifications Performed
[What you tested]

## Issues Found
### [Severity] - [Category]
**File:** \`path/to/file\`
**Problem:** [Description]
**Impact:** [What breaks]

## VERDICT
\`\`\`
PASS: All checks passed
PARTIAL: N issues found (see above)
FAIL: Critical issues prevent merge
\`\`\`

ANTI-PATTERNS TO AVOID:
- "This looks fine" (without checking)
- Skipping edge cases
- Assuming happy paths
- Not running the actual tests`;
  },
};
```

## 7.5 ClaudeCodeGuideAgent（使用指南）

```
// built-in/claudeCodeGuideAgent.ts
// 动态构建系统提示词，包含真实的工具使用上下文

interface ClaudeCodeGuideParams {
  availableSkills: string[];
  availableAgents: string[];
  availableMcpServers: string[];
  availablePlugins: string[];
  installedCommands: string[];
  uses3PServices: boolean;  // 是否使用了第三方服务
}

function getClaudeCodeGuideSystemPrompt(params: ClaudeCodeGuideParams): string {
  return [
    `You help users with Claude Code (the CLI), Claude Agent SDK, and Claude API.`,
    ``,
    `## Your Context`,
    // 动态注入用户实际可用的工具
    `Available skills: ${params.availableSkills.join(', ') || 'none'}`,
    `Available agents: ${params.availableAgents.join(', ') || 'built-in only'}`,
    `Available MCP servers: ${params.availableMcpServers.join(', ') || 'none'}`,
    `Available plugins: ${params.availablePlugins.join(', ') || 'none'}`,
    `Installed CLI commands: ${params.installedCommands.join(', ')}`,
    ``,
    // 根据使用情况调整反馈风格
    params.uses3PServices
      ? `Note: For external service issues, recommend checking official docs.`
```

```
      : ``,
  ].join('\n');
}
```

### 7.6 功能开关

```
// builtInAgents.ts: 条件注册
function getBuiltInAgents(): AgentDefinition[] {
  const agents = [GENERAL_PURPOSE_AGENT, STATUSLINE_SETUP_AGENT];

  // Explore/Plan agents: 由 BUILTIN_EXPLORE_PLAN_AGENTS 功能开关控制
  if (areExplorePlanAgentsEnabled()) {
    agents.push(EXPLORE_AGENT, PLAN_AGENT);
  }

  // Verification agent: 由 GrowthBook 实验控制
  if (isVerificationAgentEnabled()) {
    agents.push(VERIFICATION_AGENT);
  }

  // Claude Code Guide: 非 SDK 环境
  if (!isSdkEntrypoint()) {
    agents.push(CLAUDE_CODE_GUIDE_AGENT);
  }

  // 协调者模式: 使用不同的 worker agents
  if (isCoordinatorMode()) {
    return getCoordinatorAgents();  // 来自 coordinatorMode.ts
  }

  return agents;
}
```

---

## 第八章：持久化记忆系统

> **Note**
>
> 详见“ 记忆系统” 的相关文档

### 8.1 三级记忆架构

```
┌──────────────────────────────────────────────────────────────────┐
│                      Agent Memory Architecture                    │
├──────────────────────────────────────────────────────────────────┤
│                                                                   │
│  scope = 'user'                                                   │
│  路径: ~/.claude/agent-memory/<agentType>/                        │
│  内容: ~/.claude/projects/-home-xxx/MEMORY.md                     │
│  访问: 跨所有项目，跨所有会话                                      │
│  用途: 用户偏好、工作风格、长期约束                                 │
│                                                                   │
```

```
│  scope = 'project'                                          │
│  路径: <cwd>/.claude/agent-memory/<agentType>/               │
│  内容: <cwd>/.claude/agent-memory/MEMORY.md                  │
│  访问: 当前项目内所有会话                                     │
│  用途: 项目上下文、团队约定、issue 追踪                       │
│                                                             │
│  scope = 'local'                                            │
│  路径: <cwd>/.claude/agent-memory-local/<agentType>/         │
│  内容: <cwd>/.claude/agent-memory-local/MEMORY.md            │
│  访问: 仅当前任务                                            │
│  用途: 一次性任务状态、临时推理结果                           │
│                                                             │
│  记忆格式:                                                   │
│  ┌──────────────────────────────────────────────────────┐   │
│  │ ---                                                  │   │
│  │ name: {{memory name}}                                │   │
│  │ description: {{one-line description}}                │   │
│  │ type: {{user, feedback, project, reference}}         │   │
│  │ ---                                                  │   │
│  │                                                      │   │
│  │ {{memory content}}                                   │   │
│  └──────────────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────────────┘
```

## 8.2 记忆加载算法

```
// agentMemory.ts: loadAgentMemoryPrompt()

function loadAgentMemoryPrompt(
  agentType: string,
  scope: AgentMemoryScope
): string {
  // Step 1: 获取记忆文件路径
  const memoryPath = getAgentMemoryEntrypoint(agentType, scope);

  // Step 2: 读取文件内容
  if (!existsSync(memoryPath)) {
    return '';  // 无记忆文件 → 返回空
  }

  const content = readFileSync(memoryPath, 'utf-8');

  // Step 3: 解析 frontmatter
  const { metadata, body } = parseFrontmatter(content);

  // Step 4: 过滤过期记忆（如果有 decay 字段）
  const validMemories = filterValidMemories(metadata, body);

  // Step 5: 构建注入文本
  return [
    `## Relevant Memory`,
    `(Scope: ${getMemoryScopeDisplay(scope)})`,
    ``,
```

```
    ...validMemories.map(m => `- ${m.description}\n  ${m.body}`),
  ].join('\n');
}

// 使用点：在 buildAgentSystemPrompt() 中
function getAgentSystemPrompt(agentDef: AgentDefinition, ...) {
  const systemPromptLines: string[] = [];

  // Agent 自带的 system prompt
  if (isBuiltInAgent(agentDef)) {
    systemPromptLines.push(agentDef.getSystemPrompt(params));
  } else {
    systemPromptLines.push(agentDef.getSystemPrompt());
  }

  // 注入记忆（如果有 memory scope）
  if (agentDef.memory) {
    const memoryPrompt = loadAgentMemoryPrompt(
      agentDef.agentType,
      agentDef.memory
    );
    if (memoryPrompt) {
      systemPromptLines.push(memoryPrompt);
    }
  }

  return systemPromptLines;
}
```

### 8.3 记忆安全路径验证

```
// agentMemory.ts: isAgentMemoryPath()

// 防止路径穿越攻击：确保记忆访问不超出边界
function isAgentMemoryPath(absolutePath: string): boolean {
  const normalized = path.normalize(absolutePath);

  // 检查三个基准路径
  const checks = [
    normalized.startsWith(path.join(os.homedir(), '.claude', 'agent-memory')),
    normalized.startsWith(path.join(process.cwd(), '.claude', 'agent-memory')),
    // 支持远程挂载路径
    normalized.startsWith(process.env.CLAUDE_CODE_REMOTE_MEMORY_DIR ?? ''),
  ];

  return checks.some(Boolean);
}
```

---

## 第九章：任务状态机与生命周期

### 9.1 完整状态机

```
                    ┌──────────────────────────┐
                    │         pending          │
                    │   (已注册，等待启动)       │
                    │   registerAsyncAgent()   │
                    └──────────────────────────┘
                                 │
                                 │
                  registerAgentForeground()
                  (同步分支注册时)
                                 │
                                 ▼
                    ┌──────────────────────────┐
                    │         running          │
                    │    (正在执行 agent)       │
                    │  runAgent() / query()    │
                    └──────────────────────────┘
                                 │
                                 │
          ┌──────────────────────┼──────────────────────┐
          │                      │                      │
   外部中止 (ESC)          超时 / maxTurns         正常完成
          │                      │                      │
          ▼                      ▼                      ▼
┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│     killed       │   │     completed    │   │     completed    │
│  (用户中止)       │   │  (含 partial     │   │  (完整结果)       │
│                  │   │   result)        │   │                  │
└──────────────────┘   └──────────────────┘   └──────────────────┘

                                 ▲
                                 │
                    异步循环异常 / 错误
                                 │
                                 ▼
                    ┌──────────────────────────┐
                    │         failed           │
                    │     (执行异常)            │
                    │  runAsyncAgentLifecycle  │
                    └──────────────────────────┘
```

## 9.2 通知 XML 格式

```xml
<!-- enqueueAgentNotification() 生成格式 -->

<task-notification>
  <task-id>agent-7f3a9c</task-id>
  <output-file>
    /home/XXXX/.claude/agent-transcripts/agent-7f3a9c.jsonl
  </output-file>
  <tool-use-id>agent-tool-use-7f3a9c</tool-use-id>
  <status>completed</status>
  <summary>Agent "Explore" completed</summary>
  <result>
    Found 12 TypeScript files matching the pattern:
    - src/auth/login.ts
```

```
    - src/auth/session.ts
    ...
  </result>
  <usage>
    <total_tokens>4820</total_tokens>
    <input_tokens>1204</input_tokens>
    <output_tokens>3616</output_tokens>
    <tool_uses>7</tool_uses>
    <duration_ms>8430</duration_ms>
    <cache_creation_input_tokens>14200</cache_creation_input_tokens>
    <cache_read_input_tokens>3200</cache_read_input_tokens>
  </usage>
  <!-- 仅在 worktree 隔离时出现 -->
  <worktree>
    <worktree-path>
      /repo/.claude/worktrees/agent-7f3a9c
    </worktree-path>
    <worktree-branch>claude/agent-7f3a9c</worktree-branch>
  </worktree>
</task-notification>
```

## 9.3 GC 驱逐策略

当 Agent 任务完成后，系统需要决定何时将其从内存中清除（驱逐），以避免资源泄漏。

驱逐需要同时满足四个条件：

- 条件 1：retain 为 false，即 UI 层已不再持有该任务
- 条件 2：diskLoaded 为 true，即任务已从磁盘加载过（bootstrap 已完成）
- 条件 3：任务状态为 completed / failed / killed（已完成 / 失败 / 被杀死）
- 条件 4：当前时间已超过驱逐截止时间 evictAfter

截止时间的设置策略根据任务类型区分：

- 后台任务（用户在后台运行的 Agent）：允许存活 1 小时，方便随时切回查看
- 已读取的前台任务：5 分钟后即可驱逐，因为用户已经看过结果
- 活跃任务：永不驱逐，evictAfter 保持 undefined

```typescript
// LocalAgentTask.tsx: 任务回收算法

function shouldEvictTask(task: LocalAgentTaskState, now: number): boolean {
  return (
    // 条件 1: UI 不持有
    !task.retain &&
    // 条件 2: 已读取过（bootstrap 完成）
    task.diskLoaded &&
    // 条件 3: 已完成 / 失败 / 被杀
    (task.status === 'completed' ||
     task.status === 'failed' ||
     task.status === 'killed') &&
    // 条件 4: 超过驱逐截止时间
    (task.evictAfter ? now > task.evictAfter : false)
  );
}
```

```
// evictAfter 设置策略
function setEvictDeadline(task: LocalAgentTaskState): number | undefined {
  if (task.isBackgrounded) {
    // 后台任务：1 小时后可以驱逐
    return Date.now() + 60 * 60 * 1000;
  }
  if (task.retrieved) {
    // 已读取的前台任务：5 分钟后驱逐
    return Date.now() + 5 * 60 * 1000;
  }
  return undefined;  // 活跃任务不驱逐
}
```

## 9.4 进度追踪算法

```
// LocalAgentTask.tsx: updateProgressFromMessage()

function updateProgressFromMessage(
  tracker: ProgressTracker,
  message: Message,
  resolveActivityDescription: ActivityDescriptionResolver,
  tools?: Tools,
): void {
  if (message.type !== 'assistant') return;

  // Step 1: 更新 token 计数
  tracker.latestInputTokens = message.usage?.input_tokens
    ?? tracker.latestInputTokens;
  tracker.cumulativeOutputTokens +=
    message.usage?.output_tokens ?? 0;

  // Step 2: 提取 tool_use blocks
  const toolUses = extractToolUseBlocks(message);
  for (const toolUse of toolUses) {
    tracker.toolUseCount++;

    // Step 3: 分类活动强度
    const isSearch = tools?.isSearchTool(toolUse.name);
    const isRead = tools?.isReadTool(toolUse.name);

    // Step 4: 构建活动描述
    const activity: ToolActivity = {
      toolName: toolUse.name,
      input: toolUse.input,
      activityDescription: resolveActivityDescription(message, toolUse.name),
      isSearch: isSearch ?? false,
      isRead: isRead ?? false,
    };

    tracker.recentActivities.push(activity);

    // Step 5: 限制队列长度（保持最近 5 条）
```

```
    if (tracker.recentActivities.length > 5) {
      tracker.recentActivities.shift();
    }
  }
}

// getProgressUpdate: 生成进度快照
function getProgressUpdate(tracker: ProgressTracker): AgentProgress {
  const recentNonSearch = tracker.recentActivities
    .filter(a => !a.isSearch)
    .slice(-3);

  // 生成摘要文本
  const summary = recentNonSearch.length > 0
    ? `Last: ${recentNonSearch.map(a =>
        `${a.toolName}(${truncate(JSON.stringify(a.input), 50)})`
      ).join(' → ')}`
    : undefined;

  return {
    toolUseCount: tracker.toolUseCount,
    tokenCount: tracker.latestInputTokens + tracker.cumulativeOutputTokens,
    lastActivity: tracker.recentActivities.at(-1),
    recentActivities: tracker.recentActivities,
    summary,
  };
}
```

---

# 第十章：隔离机制

## 10.1 Worktree 隔离

```
// 工作流程
// 1. AgentTool.call() 检测到 isolation: 'worktree'
// 2. 创建 git worktree
// 3. fork subagent 在 worktree 中运行
// 4. 完成后通知父 session

// 创建 worktree
const worktreePath = path.join(
  repoRoot,
  '.claude',
  'worktrees',
  `agent-${agentId}`
);

const worktreeBranch = `claude/agent-${agentId}`;

// git worktree add <path> <branch> -b <new-branch>
await execGit(['worktree', 'add', '-B', worktreeBranch, worktreePath]);

// 克隆文件读取缓存（保持父的 cache hit 信息）
```

```
const clonedReadCache = cloneFileStateCache(
  toolUseContext.readFileState,
  {
    fromPath: repoRoot,
    toPath: worktreePath,
  }
);

// 在 worktree 中运行 agent
const worktreeToolUseContext = {
  ...toolUseContext,
  cwd: worktreePath,
  readFileState: clonedReadCache,
};

// 完成后处理
async function handleWorktreeCompletion(worktreePath: string, hasChanges: boolean) {
  if (!hasChanges) {
    // 无变更：自动清理 worktree
    await execGit(['worktree', 'remove', worktreePath, '--force']);
    return undefined;
  }

  // 有变更：保留 worktree，通知用户路径
  return {
    worktreePath,
    worktreeBranch,
    message: `Agent completed. Changes in: ${worktreePath}`,
  };
}
```

### 10.2 Remote 隔离（CCR only）

```
// isolation: 'remote' 仅在 ant 内部版本可用
// 通过 CCR (Claude Code Remote) 协议将 agent 调度到远程机器执行

interface RemoteAgentParams {
  prompt: string;
  agentType: string;
  model?: string;
  remoteHost: string;
  credentials: AgentCredentials;
  // ... 其他参数
}

// 流程：远程 agent 在远程机器运行
// 结果通过 CCR 协议异步传回本地
// 本地收到 <task-notification> 格式与本地异步 agent 相同
```

---

## 第十一章：协调者多 Agent 编排

> **Note**
>
> 详见 “Agent Team” 的相关文档

## 11.1 协调者模式架构

```
┌──────────────────────────────────────────────────────────────────────┐
│                    Coordinator Mode Architecture                     │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │                Coordinator Agent                             │   │
│   │  Role: Orchestrator（编排者，不亲自执行）                      │   │
│   │  Tools: Agent, SendMessage, TaskStop, Read, Bash, ...        │   │
│   │  System Prompt: coordinatorMode.ts                           │   │
│   └──────────────────────────────────────────────────────────────┘   │
│                              │                                       │
│                              │                                       │
│        spawn worker          │          spawn worker                 │
│            ┌─────────────────┴─────────────────┐                     │
│            ▼                                   ▼                     │
│   ┌────────────────────┐              ┌────────────────────┐         │
│   │  Worker 1          │              │  Worker 2          │         │
│   │  type: "worker"    │              │  type: "worker"    │         │
│   │  Tools: 受限集      │              │  Tools: 受限集      │         │
│   │  Agent Teams: ✓    │              │  Agent Teams: ✓    │         │
│   └────────────┬───────┘              └────────────┬───────┘         │
│                │  task-notification                 │                 │
│                └────────────────────────────────────┘                 │
│                              │                                       │
│                              ▼                                       │
│                Coordinator synthesizes results                       │
│                and responds to user                                  │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

## 11.2 协调者系统提示词关键段落

```
// coordinatorMode.ts: getCoordinatorSystemPrompt()

// 核心角色定义
const coordinatorPrompt = [
  `You are a task orchestration coordinator.`,
  `Your role is to PLAN, ASSIGN, and SYNTHESIZE — not to execute directly.`,

  // Worker spawn 决策矩阵
  `USE SPAWN WHEN:`,
  `   - Task requires different tool sets (search vs edit)`,
  `   - Independent subtasks that can run in parallel`,
  `   - Specialized expertise (frontend, backend, infra)`,

  `USE CONTINUE WHEN:`,
  `   - Next step logically depends on current step's result`,
```

```
  `   - Context overlap > 60% with current task`,
  `   - Subtask is a natural continuation`,

  // Worker 提示词质量指南
  `SPAWNING BEST PRACTICES:`,
  `   - Write self-contained prompts: include all context the worker needs`,
  `   - Be specific about expected output format`,
  `   - Include constraints and edge cases`,
  `   - Keep prompts under 500 words for best results`,
  `   - Synthesize worker results: don't just concatenate`,

  // 通知格式说明
  `HOW WORKER RESULTS ARRIVE:`,
  `   Worker results come as <task-notification> messages.`,
  `   These are user-role messages — read them to get worker output.`,
].join('\n');
```

## 11.3 Worker 工具上下文

```
// coordinatorMode.ts: getCoordinatorUserContext()
// 为 worker agent 构建受限的工具上下文

function getCoordinatorUserContext(
  mcpClients: McpClients,
  scratchpadDir?: string,
): Record<string, string> {
  const tools: string[] = [
    // 核心工具
    'Read', 'Glob', 'Grep', 'WebSearch', 'WebFetch',
    'Write', 'Edit', 'Bash', 'NotebookEdit',
    // 任务工具
    'TodoWrite', 'TodoGet', 'TodoList', 'TodoUpdate',
    // 团队通信
    'SendMessage',
    // 搜索增强
    'ToolSearch',
  ];

  // 如果有 scratchpad（跨 worker 持久化目录），包含其路径
  if (scratchpadDir) {
    tools.push(
      `NOTE: Scratchpad at ${scratchpadDir} — write intermediate findings there.`
    );
  }

  return {
    available_tools: tools.join(', '),
    mcp_servers: Object.keys(mcpClients ?? {}).join(', ') || 'none',
  };
}
```

# 第十二章：数据流全景图

```
User Terminal
    │
    │ "Analyze the auth module and fix the session bugs"
    │
    ▼
┌──────────────────────────────────────────────────────────────────────┐
│                        Main Session                                  │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  Claude (Main)                                               │   │
│  │ thinking: This is complex. Let me spawn an Explore agent...  │   │
│  │                                                              │   │
│  │  Tools:                                                      │   │
│  │  ┌──────────────────────────────────────────────────────┐   │   │
│  │  │  AgentTool.call({                                    │   │   │
│  │  │    prompt: "Find all auth/session related files",    │   │   │
│  │  │    subagent_type: "Explore",                         │   │   │
│  │  │  })                                                  │   │   │
│  │  └──────────────────────────────────────────────────────┘   │   │
│  └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────┬───────────────────────────────────────┘
                               │
          ┌────────────────────┼──────────────────────────────┐
          │ fork path          │ normal path                  │
          ▼                    ▼                              ▼
┌──────────────────┐  ┌──────────────────┐        ┌──────────────────┐
│  Fork Subagent   │  │  Explore Agent   │        │  [Other Agent]   │
│  (隔离 worktree)  │  │  (同步 one-shot)  │        │  (异步后台)       │
│                  │  │                  │        │                  │
│  Inherits:       │  │  useExactTools ✓ │        │ registerAsync    │
│  - system prompt │  │  forkCtx ✓       │        │ Agent()          │
│  - tool defs     │  │                  │        │                  │
│  - file cache    │  │  for await msg   │        │ runAsync         │
│                  │  │  of query()      │        │ AgentLifecycle() │
│  buildForked     │  │  → yield msg     │        │  ↓               │
│  Messages()      │  │                  │        │  enqueue         │
│  ↓               │  │  return result   │        │  Notification()  │
│  query()         │  │                  │        │                  │
│  (cache hit ✓)   │  └──────────┬───────┘        └──────────┬───────┘
│                  │             │                           │
│                  │             ▼                           ▼
│                  │  ┌──────────────────────────────────────────┐
│                  │  │         Results Aggregation              │
│                  │  │                                          │
│                  │  │  AgentToolResult {                       │
│                  │  │    agentId: "agent-abc",                 │
│                  │  │    content: "Found 12 files: ...",       │
│                  │  │    totalToolUseCount: 7,                 │
│                  │  │    totalTokens: 4820,                    │
│                  │  │    totalDurationMs: 8430,                │
```

```
│                  │  │    usage: { input_tokens: 1204, ... }       │
│                  │  │  }                                           │
│                  │  └──────────────────────────────────────────────┘
│                  │           │
└──────────────────┴───────────┴──────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────────┐
│                     Main Session (continued)                         │
│                                                                      │
│  Agent result received. Now fixing the bugs...                       │
│                                                                      │
│  Another AgentTool.call() → spawn general-purpose agent             │
│  (with verification) to make the fixes                               │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  <task-notification> (from background agents)               │   │
│  │  When async agents complete, their results appear here      │   │
│  │  as user-role messages in the main conversation              │   │
│  └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
```

---

# 第十三章：关键算法伪代码实现

## 13.1 工具过滤算法

```
ALGORITHM: filterToolsForAgent
INPUT:
  - allTools: Tool[]          全部可用工具
  - agentDef: AgentDefinition  Agent 定义
  - isAsync: boolean           是否异步执行
  - permissionMode: PermissionMode 权限模式

OUTPUT: filteredTools: Tool[]

BEGIN
  filtered ← allTools

  # Layer 1: 全局禁止列表
  filtered ← filtered EXCEPT ALL_AGENT_DISALLOWED_TOOLS
  IF USER_TYPE ≠ 'ant' THEN
    filtered ← filtered EXCEPT 'Agent'
  END IF

  # Layer 2: 异步强制白名单
  IF isAsync = TRUE THEN
    filtered ← filtered INTERSECT ASYNC_AGENT_ALLOWED_TOOLS
  END IF

  # Layer 3: 自定义 Agent 额外禁止
  IF agentDef.source ∈ {userSettings, projectSettings} THEN
    filtered ← filtered EXCEPT CUSTOM_AGENT_DISALLOWED_TOOLS
```

```
  END IF

  # Layer 4: Agent 允许列表
  IF agentDef.tools EXISTS THEN
    IF agentDef.tools ≠ ['*'] THEN
      filtered ← filtered INTERSECT agentDef.tools
    END IF
  END IF

  # Layer 5: Agent 禁止列表（优先级最高）
  IF agentDef.disallowedTools EXISTS THEN
    filtered ← filtered EXCEPT agentDef.disallowedTools
  END IF

  # Layer 6: 权限模式调整
  SWITCH permissionMode:
    CASE 'bypassPermissions':
      filtered ← filtered.map(t → t.removePermissionWrapper())
    CASE 'acceptEdits':
      filtered ← filtered.map(t →
        IF t.name ∈ {Edit, Write} THEN t.withAutoApprove() ELSE t
      )
    CASE 'plan':
      filtered ← filtered.map(t → t.wrapWithPlanApproval())
  END SWITCH

  RETURN filtered
END
```

## 13.2 Fork 消息构建算法

```
ALGORITHM: buildForkedMessages
INPUT:
  - directive: string           子 Agent 的任务指令
  - assistantMessage: Message  父 Agent 的最后一条 assistant 消息

OUTPUT: forkedMessages: Message[]

BEGIN
  # Step 1: 深拷贝（避免污染父的 REPL state）
  cloned ← DEEP_CLONE(assistantMessage)

  # Step 2: 提取 tool_use blocks
  toolUses ← cloned.content WHERE block.type = 'tool_use'

  # Step 3: 为每个 tool_use 创建字节级一致的占位结果
  # 关键: FORK_PLACEHOLDER_RESULT 是常量字符串
  # API 的 tool_result 序列化结果完全相同 → cache hit
  toolResultBlocks ← []
  FOR EACH toolUse IN toolUses DO
    APPEND toolResultBlocks ← {
      type: 'tool_result',
      tool_use_id: toolUse.id,
```

```
      content: [{
        type: 'text',
        text: FORK_PLACEHOLDER_RESULT,
        is_error: false,
      }]
    }
  END FOR

  # Step 4: 构建用户指令消息（新增，不参与 cache）
  directiveMsg ← {
    type: 'user',
    content: [{
      type: 'text',
      text: buildChildMessage(directive),  # 10 条 FORK_BOILERPLATE_TAG 规则
    }]
  }

  # Step 5: 组装消息序列
  RETURN [cloned] + toolResultBlocks + [directiveMsg]
END

FUNCTION buildChildMessage(directive)
  RETURN "<FORK_BOILERPLATE_TAG>\n" +
    "Task: " + directive + "\n" +
    "Rules: " +
    "1. Continue from previous agent's last action\n" +
    "2. Complete the task autonomously\n" +
    "3. Report completion with concise summary\n" +
    "4. For multi-step: execute all remaining steps\n" +
    "5. One-shot: run to completion without asking\n" +
    "6. Tool restrictions override your default capabilities\n" +
    "7. If blocked: attempt alternatives\n" +
    "8. Do not explain reasoning unless asked\n" +
    "9. Result goes to parent session\n" +
    "10. You control execution — parent is waiting\n" +
    "</FORK_BOILERPLATE_TAG>"
END FUNCTION
```

## 13.3 进度追踪算法

```
ALGORITHM: updateProgressFromMessage
INPUT:
  - tracker: ProgressTracker     运行时追踪器
  - message: Message              来自 query() 的消息
  - resolveActivityDescription:  描述解析函数
  - tools: Tool[]                 可用工具集

BEGIN
  IF message.type ≠ 'assistant' THEN
    RETURN  # 仅处理 assistant 消息
  END IF

  # Step 1: 更新 token 计数
```

```
  IF message.usage EXISTS THEN
    tracker.latestInputTokens ← message.usage.input_tokens
    tracker.cumulativeOutputTokens ←
      tracker.cumulativeOutputTokens + message.usage.output_tokens
  END IF

  # Step 2: 处理 tool_use blocks
  toolUses ← EXTRACT_TOOL_USE_BLOCKS(message)
  FOR EACH toolUse IN toolUses DO
    tracker.toolUseCount ← tracker.toolUseCount + 1

    # Step 3: 分类工具强度（搜索=低强度，不计入"活跃"）
    isSearch ← tools?.isSearchTool(toolUse.name) ?? FALSE
    isRead ← tools?.isReadTool(toolUse.name) ?? FALSE

    # Step 4: 构建活动记录
    activity ← {
      toolName: toolUse.name,
      input: toolUse.input,
      activityDescription: resolveActivityDescription(
        message, toolUse.name
      ),
      isSearch,
      isRead,
    }

    # Step 5: 追加到最近活动队列（最多 5 条）
    APPEND tracker.recentActivities ← activity
    IF LENGTH(tracker.recentActivities) > 5 THEN
      SHIFT tracker.recentActivities  # 删除最早的
    END IF
  END FOR
END

ALGORITHM: getProgressUpdate
INPUT: tracker: ProgressTracker

OUTPUT: AgentProgress

BEGIN
  # 生成摘要：取最近的非搜索活动
  recentNonSearch ← FILTER(
    tracker.recentActivities,
    a → NOT a.isSearch
  )[-3:]  # 最后 3 条

  summary ← ""
  IF LENGTH(recentNonSearch) > 0 THEN
    summary ← "Last: " + JOIN(
      recentNonSearch.map(a →
        a.toolName + "(" + TRUNCATE(JSON.stringify(a.input), 50) + ")"
      ),
      " → "
```

```
    )
  END IF

  RETURN {
    toolUseCount: tracker.toolUseCount,
    tokenCount: tracker.latestInputTokens + tracker.cumulativeOutputTokens,
    lastActivity: LAST(tracker.recentActivities),
    recentActivities: tracker.recentActivities,
    summary: summary IF summary ≠ "" ELSE undefined,
  }
END
```

---

# 第十四章：设计模式与架构哲学

## 14.1 工厂模式：Agent Definition Registry

```
// builtInAgents.ts
// 统一的 agent 获取入口，支持功能开关和动态注册

class AgentRegistry {
  private agents: Map<string, AgentDefinition> = new Map();

  register(agent: AgentDefinition): void {
    // 同类型覆盖（built-in 优先于 custom）
    if (agent.source === 'built-in' || !this.agents.has(agent.agentType)) {
      this.agents.set(agent.agentType, agent);
    }
  }

  get(type: string): AgentDefinition | undefined {
    return this.agents.get(type);
  }

  getAll(): AgentDefinition[] {
    return [...this.agents.values()];
  }
}

// 使用点
function resolveAgent(agentType: string): AgentDefinition {
  return (
    builtInAgents.get(agentType) ??
    customAgents.get(agentType) ??
    generalPurposeAgent  // 默认回退
  );
}
```

## 14.2 策略模式：工具过滤策略

```
// 不同的 agent 类型 → 不同的过滤策略
const filterStrategies = {
```

```
  async:      (tools, agentDef) => filterToolsAsync(tools, agentDef),
  builtin:    (tools, agentDef) => filterToolsBuiltin(tools, agentDef),
  custom:     (tools, agentDef) => filterToolsCustom(tools, agentDef),
  coordinator:(tools, agentDef) => filterToolsCoordinator(tools, agentDef),
};

// runAgent.ts 中根据类型分发
const strategy = filterStrategies[agentCategory] ?? filterStrategies.builtin;
const filteredTools = strategy(allTools, agentDef);
```

## 14.3 迭代器模式：消息流

```
// runAgent.ts: 核心迭代模式
// runAgent() 是 AsyncGenerator<Message>，消费 query() 的消息流

async function* runAgent(params): AsyncGenerator<Message> {
  // 前置设置...
  try {
    for await (const message of query(queryOptions)) {
      // 每个消息实时 yield 给调用方（AgentTool.call）
      yield message;
    }
  } finally {
    // 无论正常还是异常，finally 确保清理
    await cleanup();
  }
}

// AgentTool.tsx 中的消费方式
for await (const message of runAgent(params)) {
  handleMessage(message);
}
```

## 14.4 模板方法：Agent 执行骨架

```
// runAgent.ts: 定义骨架，子类/配置可定制各步骤
class AgentExecutor {
  async* execute(params: RunAgentParams): AsyncGenerator<Message> {
    await this.setup(params);           // 固定：初始化
    const options = await this.buildQueryOptions(params); // 可Override
    try {
      for await (const message of query(options)) {
        yield message;
      }
    } finally {
      await this.cleanup(params);       // 固定：清理
    }
  }

  async setup(params) { /* 初始化 */ }
  async buildQueryOptions(params) { /* 构建选项 */ }
```

```
  async cleanup(params) { /* 清理 */ }
}
```

## 14.5 装饰器模式：权限包装

```
// 工具调用被权限装饰器包装，根据权限模式动态选择行为

function wrapToolWithPermission(
  tool: Tool,
  permissionMode: PermissionMode,
  canUseTool: CanUseToolFn,
): Tool {
  switch (permissionMode) {
    case 'bypassPermissions':
      // 零检查，直接执行
      return tool;

    case 'acceptEdits':
      // Edit/Write 自动批准，其他正常审批
      if (tool.name === 'Edit' || tool.name === 'Write') {
        return tool.withAutoApprove();
      }
      return tool.withPermissionCheck(canUseTool);

    case 'plan':
      // 所有工具都需要先出 plan
      return tool.withPlanApproval();

    case 'bubble':
    default:
      // 正常权限检查，向上冒泡
      return tool.withPermissionCheck(canUseTool);
  }
}
```

## 14.6 发布-订阅模式：进度通知

```
// LocalAgentTask.tsx: ProgressTracker 实现发布-订阅
class ProgressTracker {
  private listeners: Array<(progress: AgentProgress) => void> = [];
  private tracker: ProgressTracker;  // 内部状态

  // 订阅
  subscribe(listener: (progress: AgentProgress) => void): () => void {
    this.listeners.push(listener);
    // 返回取消订阅函数
    return () => {
      this.listeners = this.listeners.filter(l => l !== listener);
    };
  }

  // 发布
```

```
  private notify(): void {
    const progress = getProgressUpdate(this.tracker);
    this.listeners.forEach(listener => listener(progress));
  }

  // 更新时自动发布
  update(message: Message, ...args): void {
    updateProgressFromMessage(this.tracker, message, ...args);
    this.notify();  // 自动通知所有订阅者
  }
}
```

### 14.7 命令模式：Task 生命周期

```
// LocalAgentTask.tsx: 每个 task 操作都是不可变的状态转换
type TaskCommand =
  | { type: 'start'; payload: RunAgentParams }
  | { type: 'background'; payload: { taskId } }
  | { type: 'complete'; payload: { result: AgentToolResult } }
  | { type: 'fail'; payload: { error: string } }
  | { type: 'kill' }
  | { type: 'discard' };

// 状态机（使用 Immer 或类似不可变更新）
function reduceTaskState(
  state: LocalAgentTaskState,
  command: TaskCommand
): LocalAgentTaskState {
  switch (command.type) {
    case 'background':
      return { ...state, isBackgrounded: true };
    case 'complete':
      return { ...state, status: 'completed', result: command.payload.result };
    case 'fail':
      return { ...state, status: 'failed', error: command.payload.error };
    case 'kill':
      return { ...state, status: 'killed' };
    default:
      return state;
  }
}
```

---

## 第十五章：安全性与边界控制

### 15.1 安全边界总览

```
┌──────────────────────────────────────────────────────────────────┐
│                        Security Boundary                         │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │                        User Session                      │   │
```

```
│   │                                                          │    │
│   │   Tool Call Boundary (AgentTool.call)                     │    │
│   │   ├── canUseTool('Agent') — 权限验证                       │    │
│   │   ├── 工具过滤 — 移除危险工具集                              │    │
│   │   ├── MCP 服务器验证 — requiredMcpServers                   │    │
│   │   ├── 递归深度守卫 — 防止 fork 嵌套                          │    │
│   │   └── Abort 传播链 — 父中止 → 所有子中止                     │    │
│   │                                                          │    │
│   │   ┌──────────────────────────────────────────────────┐   │    │
│   │   │          Sub-Agent Sandbox                        │   │    │
│   │   │                                                   │   │    │
│   │   │   • 独立 AbortController（链接到父）               │   │    │
│   │   │   • Worktree 隔离（可选，文件修改不污染父）         │   │    │
│   │   │   • 克隆的文件读取缓存（不泄露父的临时文件）        │   │    │
│   │   │   • 独立的 MCP 客户端（不共享会话级 MCP）           │   │    │
│   │   │   • 独立的 Frontmatter Hooks（不继承父的 hooks）    │   │    │
│   │   │                                                   │   │    │
│   │   │   ┌──────────────────────────────────────┐        │   │    │
│   │   │   │  Tool Execution Sandboxing            │        │   │    │
│   │   │   │                                       │        │   │    │
│   │   │   │  Edit / Write: permissionMode 控制    │        │   │    │
│   │   │   │  Bash: shell 权限模式控制             │        │   │    │
│   │   │   │  WebSearch/Fetch: 隔离的 HTTP 客户端  │        │   │    │
│   │   │   │  MCP: 独立连接，不继承父会话的 MCP 状态 │        │   │    │
│   │   │   │                                       │        │   │    │
│   │   │   └──────────────────────────────────────┘        │   │    │
│   │   └──────────────────────────────────────────────────┘   │    │
│   └──────────────────────────────────────────────────────────┘    │
│                                                                    │
│  Post-Execution Security Review:                                   │
│   ├── classifyHandoffIfNeeded() — 检查 agent 的工具使用行为        │
│   ├── 敏感文件访问审查 — agent 是否访问了不当文件                   │
│   └── 异常工具调用审查 — agent 是否绕过了工具限制                   │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

## 15.2 Handoff 安全分类

```
// agentToolUtils.ts: classifyHandoffIfNeeded()
// 在 agent 完成后执行，用于审查子 agent 的行为

async function classifyHandoffIfNeeded(params: {
  agentId: string;
  result: AgentToolResult;
  toolUseContext: ToolUseContext;
}): Promise<string | null> {
  // 功能开关：默认关闭
  if (!isHandoffClassifierEnabled()) {
    return null;
  }

  // 收集 agent 的所有工具调用
  const allToolCalls = extractAllToolCalls(params.result.agentMessages);
```

```
  // 审查维度
  const concerns: string[] = [];

  // 1. 检查是否访问了敏感路径
  const sensitivePaths = [
    process.env.HOME + '/.ssh/',
    process.env.HOME + '/.aws/',
    '/etc/secrets/',
  ];
  for (const call of allToolCalls) {
    if (call.tool === 'Read' || call.tool === 'Glob') {
      for (const path of sensitivePaths) {
        if (call.input.path?.startsWith(path)) {
          concerns.push(`Sensitive path access: ${path}`);
        }
      }
    }
  }

  // 2. 检查是否有大量写操作（可能异常）
  const writeCount = allToolCalls.filter(
    c => c.tool === 'Edit' || c.tool === 'Write'
  ).length;
  if (writeCount > 50) {
    concerns.push(`Unusually high write count: ${writeCount}`);
  }

  // 3. 检查是否使用了非预期工具
  const allowedTools = ASYNC_AGENT_ALLOWED_TOOLS;
  for (const call of allToolCalls) {
    if (!allowedTools.has(call.tool)) {
      concerns.push(`Unexpected tool: ${call.tool}`);
    }
  }

  // 返回分类结果
  if (concerns.length === 0) {
    return 'CLEAR';
  }
  return concerns.join('; ');
}
```

## 15.3 Abort 传播链

```
// forkedAgent.ts: createSubagentContext()
// Subagent 的 AbortController 链接到父的 AbortController

function createChildAbortController(
  parentController: AbortController | undefined,
  agentId: string,
): AbortController {
  const child = new AbortController();
```

```
  if (parentController) {
    // 父 abort → 自动触发子的 abort
    parentController.signal.addEventListener('abort', () => {
      child.abort(`Parent session cancelled (agent: ${agentId})`);
    }, { once: true });
  }

  return child;
}

// 使用场景
// 1. 用户按 ESC → 主 session abort → 传播到所有子 agent
// 2. 父 agent 完成 → 中止还在运行的子 agent
// 3. 工具超时 → abort 当前 agent
```

### 15.4 路径穿越防护

```
// agentMemory.ts: isAgentMemoryPath()
// 确保 agent 无法通过 ../../../ 访问记忆目录外的文件

function isAgentMemoryPath(absolutePath: string): boolean {
  const normalized = path.normalize(absolutePath);

  const memoryBase = path.join(os.homedir(), '.claude', 'agent-memory');
  const projectMemoryBase = path.join(process.cwd(), '.claude', 'agent-memory');
  const remoteMemoryBase = process.env.CLAUDE_CODE_REMOTE_MEMORY_DIR ?? '';

  return (
    normalized.startsWith(memoryBase) ||
    normalized.startsWith(projectMemoryBase) ||
    (remoteMemoryBase !== '' && normalized.startsWith(remoteMemoryBase))
  );
}

// 使用点：在 agentMemory.ts 的所有文件读取中
function loadAgentMemoryPrompt(agentType: string, scope: AgentMemoryScope) {
  const memoryPath = getAgentMemoryEntrypoint(agentType, scope);

  // 安全检查：防止通过 ../../../etc/passwd 绕过
  if (!isAgentMemoryPath(memoryPath)) {
    throw new Error(`Security: attempted memory path traversal: ${memoryPath}`);
  }

  return readFileSync(memoryPath, 'utf-8');
}
```

---

## 附录：常量速查表

```
// ==================== 工具名称 ====================
AGENT_TOOL_NAME           = 'Agent'
```

```
LEGACY_AGENT_TOOL_NAME    = 'Task'

// ==================== Agent 类型 ====================
FORK_SUBAGENT_TYPE        = 'fork'          // Fork subagent 虚拟类型
WORKER_AGENT_TYPE         = 'worker'        // Coordinator worker 类型

// ==================== One-shot Agents ====================
// 这些 agent 不需要 SendMessage trailer，直接返回
ONE_SHOT_BUILTIN_AGENT_TYPES = new Set(['Explore', 'Plan'])

// ==================== Fork 常量 ====================
FORK_BOILERPLATE_TAG      = 'FORK_BOILERPLATE'
FORK_PLACEHOLDER_RESULT   = 'Fork started — processing in background'

// ==================== 时间阈值 ====================
PROGRESS_THRESHOLD_MS     = 2000            // 显示后台提示的延迟（ms）
AUTO_BACKGROUND_MS        = 120_000         // 自动后台化的超时（ms，默认）
DEFAULT_AGENT_TIMEOUT_MS  = 600_000         // 默认 agent 超时（10 分钟）

// ==================== 内存管理 ====================
MAX_RECENT_ACTIVITIES     = 5               // 进度追踪保留最近活动数
MAX_MEMORY_INDEX_LINES    = 200             // MEMORY.md 索引最大行数
DEFAULT_EVICT_AFTER_MS    = 5 * 60 * 1000   // 前台任务 5 分钟后可驱逐
BG_EVICT_AFTER_MS         = 60 * 60 * 1000  // 后台任务 1 小时后可驱逐

// ==================== Token 限制 ====================
DEFAULT_MAX_OUTPUT_TOKENS = 8192            // 默认最大输出 token
MAX_OUTPUT_TOKENS_HAIKU  = 4096            // Haiku 模型限制
```